

Evaluating Lightweight Machine-Learning Models for Detecting Rare Cyberattacks in Imbalanced IoT Network Traffic

Amizhthini Rajeshvarman

Student ID: s225593298

SIT723 Research Techniques & Applications

Deakin University

Supervisor: Dr Atul Sajjanhar

September 2026

Abstract

IoT intrusion-detection datasets are often highly imbalanced, which can make rare cyberattacks difficult to detect even when overall classification performance appears strong. At the same time, IoT and edge environments can have limited computational resources, creating a need for intrusion-detection models that provide useful detection performance without excessive computational cost. This research evaluates Logistic Regression, Decision Tree, Random Forest and LightGBM for detecting rare cyberattacks in imbalanced IoT network traffic using the CICIoT2023 dataset.

The original dataset contained more than 45 million records across 34 classes. A smaller working dataset was prepared for computational feasibility while preserving rare attack classes. Ten rare attack classes were identified using an operational threshold of less than 0.1% of the original dataset. The models were evaluated under no balancing, class weighting and SMOTE, with random undersampling also tested for Decision Tree. Precision, recall, F1-score, macro F1-score, and rare-class average F1-score were used to gauge performance. To assess computing efficiency, training time, inference time, and model size were also noted.

Random Forest achieved the strongest rare-class detection, with SMOTE producing a rare-class average F1-score of 0.4663. However, this configuration produced a model of approximately 14.32 GB. Random Forest with class weighting achieved a very similar rare-class F1-score of 0.4655 with substantially lower computational cost. LightGBM with SMOTE achieved a rare-class F1-score of 0.4167 while maintaining a model size of approximately 11.38 MB. Logistic Regression produced the smallest models but showed weaker rare-class detection.

The results show that rare-attack detection and computational efficiency need to be considered together. Higher detection performance can require substantially greater computational resources, and the most suitable approach therefore depends on the balance required between detection quality and efficiency.

1. Introduction

1.1 Background

The Internet of Things (IoT) connects a large number of physical devices that collect, exchange and process data through networks. These devices are increasingly used in homes, healthcare, transportation, industry and other environments, which also increases the amount of network traffic that must be protected from cyberattacks. Machine-learning-based intrusion detection has become an important approach for identifying malicious network activity because it can learn patterns from traffic data and classify different types of behaviour. However, IoT intrusion-detection datasets are often

highly imbalanced, meaning some attack classes appear much more frequently than others. This can make rare attacks harder to detect. At the same time, many IoT devices and edge systems have limited computing resources, creating a need for models that are not only effective but also computationally lightweight.

1.2 Research Problem

IoT environments generate large amounts of network traffic and can be exposed to many different types of cyberattacks. In intrusion-detection datasets, however, some attack classes may contain a very large number of samples while other attack classes occur only rarely. This class imbalance can cause machine-learning models to perform well overall while still failing to detect minority attack classes effectively. At the same time, IoT systems often have limited computational resources, so intrusion-detection models should not only be accurate but also efficient enough to be considered lightweight. Therefore, there is a need to evaluate machine-learning models by considering both rare-attack detection performance and computational efficiency.

1.3 Research Gap

Existing IoT intrusion-detection studies often report strong overall classification performance, but overall accuracy alone may not show whether rare attack classes are being detected effectively. Some studies also focus mainly on improving predictive performance without giving equal attention to computational requirements such as training time, inference speed and model size. In addition, class imbalance can significantly affect minority-class detection, yet different imbalance-handling methods may influence models in different ways. This leaves a void for a study that assesses lightweight machine-learning models by taking into account computational efficiency, class-imbalance handling, and rare-attack detection all within the same experimental environment.

1.4 Research Aim

The aim of this research is to evaluate the effectiveness and computational efficiency of lightweight machine-learning models for detecting rare cyberattacks in imbalanced IoT network traffic.

1.5 Research Objectives

The objectives of this research are:

1. To analyse the class distribution of the CICIoT2023 dataset and identify rare attack classes.
2. To evaluate several lightweight machine-learning models for multiclass IoT intrusion detection.

3. To investigate the effect of different class-imbalance handling methods on rare-attack detection performance.
4. To compare the models using precision, recall, F1-score and macro-F1.
5. To compare computational efficiency using measures like training time, inference time and model size.
6. To examine the trade-off between rare-attack detection performance and computational efficiency.

1.6 Research Questions

RQ1: How well do lightweight machine-learning models detect rare cyberattacks in imbalanced IoT network traffic?

RQ2: How do different class-imbalance handling methods affect rare-attack detection performance?

RQ3: What trade-off exists between detection performance and computational efficiency for the selected lightweight machine-learning models?

1.7 Thesis Structure

There are six major chapters in this thesis. The research problem, research gap, aim, objectives, and research questions are introduced in Chapter 1. The literature on IoT intrusion detection, class imbalance, infrequent cyberattacks, and lightweight machine-learning techniques is reviewed in Chapter 2. The process for preparing datasets, choosing models, managing imbalances, and assessing performance is described in Chapter 3. The findings of the experiment are presented in Chapter 4. The results in relation to the research questions and earlier investigations are covered in Chapter 5. Chapter 6 concludes by summarizing the key conclusions, constraints, and future research objectives.

2. Literature Review

2.1 IoT Intrusion Detection and Machine Learning

The growth of the Internet of Things has increased the amount of network traffic generated by connected devices and has also introduced new cybersecurity challenges. Because IoT environments can contain many devices with different behaviours, traditional security methods may struggle to identify all forms of malicious activity. Machine-learning-based intrusion detection has therefore become a common approach for analysing network traffic and identifying suspicious patterns.

Machine-learning models can be trained using labelled network-traffic data to distinguish benign behaviour from different types of cyberattacks. Previous studies have evaluated a range of models, including Logistic Regression, Decision Tree, Random

Forest, ensemble methods and boosting-based approaches. These methods vary in their detection performance, complexity and computational requirements.

For IoT environments, detection performance alone is not enough. Models may need to operate on edge devices or systems with limited memory and processing power. This has led to increasing interest in lightweight intrusion-detection models that aim to provide useful detection performance while keeping computational cost relatively low.

2.2 Class Imbalance and Rare Cyberattacks

In intrusion-detection datasets, class imbalance is a significant problem since some attack classes may have a lot of samples while others may occur infrequently. When this happens, a machine-learning model can achieve strong overall performance by learning the majority classes well while still performing poorly on minority attacks.

This is particularly important for rare cyberattacks because these classes may represent only a very small proportion of the available traffic. If evaluation relies mainly on overall accuracy, poor detection of these minority classes can be hidden. For this reason, metrics such as recall, F1-score and macro F1-score are more useful when analysing imbalanced multiclass intrusion-detection problems.

Different techniques can be used to reduce the effect of class imbalance. Class weighting gives greater importance to mistakes made on minority classes. Random undersampling reduces the number of majority-class samples, while SMOTE creates synthetic minority-class samples. These methods can improve minority-class detection, although they can also introduce trade-offs such as information loss, additional computation or synthetic-sample quality issues.

2.3 Lightweight Machine Learning for IoT Security

In IoT intrusion detection, a model should not only detect attacks effectively but should also be practical to run on systems with limited computing resources. This is especially important for IoT and edge environments, where memory, processing power and storage may be limited.

Lightweight machine-learning models are generally expected to reduce computational cost while maintaining acceptable detection performance. Different models achieve this in different ways. Logistic Regression is simple and requires very little storage, Decision Tree provides relatively low-complexity decision rules, Random Forest can improve prediction performance by combining multiple trees but may require more memory, and LightGBM is designed to provide efficient gradient boosting for structured data.

Because each model has different strengths and computational requirements, it is useful to compare both detection performance and efficiency measures such as training time, inference time and model size. This makes it possible to evaluate the

trade-off between detecting rare attacks accurately and keeping the model computationally practical.

2.4 Review of the Selected Research Papers

Neto et al. (2023) introduced the CICIoT2023 dataset, which is the main dataset used in this research. The dataset was created using 105 real IoT devices and includes 33 different attacks together with benign traffic. Their benchmark results showed that high overall accuracy does not always mean that all individual attack classes are detected well. This is important for the current research because it supports looking at class-level and macro performance rather than relying only on accuracy.

Ismail, Dandan and Qushou (2025) compared several lightweight machine-learning approaches across the TON_IoT, WUSTL-IIOT-2021 and EdgelloTset datasets. Their study included Decision Tree, Random Forest, Bagging, Stacking and LightGBM, and specifically considered the effect of imbalanced class distributions. Their work supports the use of simpler machine-learning models for IoT and IIoT intrusion detection, while also showing that performance can change across datasets.

Almahaqeri et al. (2026) investigated an optimized gradient-boosting approach using CICIoT2023. This study is relevant because it examines boosting-based intrusion detection on the same dataset used in the current research and contributes to the comparison between predictive performance and computational efficiency.

Anbiaee, Dadkhah and Ghorbani (2025) proposed the FIGS framework for highly imbalanced IoT intrusion detection. Their method combines feature selection, GAN-based sample generation and a modified SMOTE approach to improve minority-class detection. Their results showed improvements in macro-level measures, but the method is more complex than simple class weighting or standard SMOTE. This supports the current research focus on testing simpler imbalance-handling methods that may be easier to apply in lightweight environments.

Yang, Gu and Yan (2023) focused specifically on rare-class intrusion detection. Their KGMS-IDS framework combines rare-class oversampling, a denoising stacked autoencoder and ensemble learning. Although the system improved overall rare-class detection, extremely rare classes still remained difficult to identify. The study therefore supports the use of per-class precision, recall and F1-score when evaluating rare cyberattacks.

Together, these studies show that three issues are closely connected: class imbalance, rare-attack detection and computational efficiency. The current research brings these areas together by comparing simpler machine-learning models and imbalance-handling methods using one consistent CICIoT2023 experimental setup.

2.5 Research Gap

According to the studied literature, obtaining excellent overall classification performance is a common goal of IoT intrusion-detection investigations. However, high overall performance does not necessarily mean that rare attack classes are detected effectively, especially when the dataset is highly imbalanced.

The literature also shows that rare-class detection is often improved using more complex techniques such as GAN-based oversampling, modified SMOTE, deep feature learning or ensemble frameworks. These methods can improve performance, but they may increase model complexity and computational cost.

Another gap is that computational efficiency is not always examined together with rare-class performance. Measures such as training time, inference time and model size are often reported less consistently than predictive metrics.

This research addresses these gaps by comparing Logistic Regression, Decision Tree, Random Forest and LightGBM using the same CICIoT2023 train-test split. It also compares simple imbalance-handling methods, including class weighting and SMOTE, and evaluates both rare-class detection and computational efficiency within the same experimental setup.

3. Methodology

3.1 Data Cleaning

The CICIoT2023 dataset was first checked for missing and infinite values across all 63 merged CSV files. A total of 1,000 rows contained at least one missing or infinite value out of 45,019,243 total records, representing approximately 0.0022% of the dataset. Because this proportion was extremely small, these rows were removed rather than imputed. This avoided introducing artificial values into the dataset while having a negligible effect on the overall sample size.

Duplicate rows were also identified during the initial data-quality analysis. At first, these records were kept because repeated network traffic may be genuine. Later, after creating the smaller working dataset, exact duplicate rows were removed using the pandas `drop_duplicates()` function. Records with the same feature values but different class labels were kept because they were not exact duplicates.

3.2 Dataset Description

This research uses the CICIoT2023 dataset for evaluating machine-learning-based IoT intrusion detection. The processed dataset used in this study consists of 63 merged CSV files containing a total of 45,019,243 network-traffic records. Each record contains 40 columns, including network-traffic features and a target column named Label. The

dataset contains 34 classes in total, consisting of benign traffic and 33 different cyberattack classes.

Initial analysis showed that the dataset is highly imbalanced. For example, the largest class, DDOS-ICMP_FLOOD, contains 6,893,259 records, representing approximately 15.31% of the dataset, whereas the smallest class, UPLOADING_ATTACK, contains only 1,196 records, representing approximately 0.0027%. This large difference in class frequencies makes CICIoT2023 suitable for investigating the detection of rare cyberattacks under class imbalance.

3.2.1 Identification of Rare Attack Classes

For the initial experimental analysis, an attack class was provisionally considered rare if it represented less than 0.1% of the complete dataset. Using this threshold, 10 rare attack classes were identified: DDOS-HTTP_FLOOD, DDOS-SLOWLORIS, DICTIONARYBRUTEFORCE, BROWSERHIJACKING, COMMANDINJECTION, SQLINJECTION, XSS, BACKDOOR_MALWARE, RECON-PINGSWEEP and UPLOADING_ATTACK. Together, these classes contain 88,479 records and represent approximately 0.1965% of the complete dataset. The 0.1% threshold was used as the operational definition of a rare attack class for the experiments in this study.

3.3 Working Dataset Preparation

Because the complete CICIoT2023 dataset contains more than 45 million records, running repeated experiments on the full dataset was not practical on the available computer. Therefore, a smaller working dataset was created using Python. Common classes were reduced, while smaller classes were kept as much as possible so that rare attacks remained available for testing. The working dataset initially contained 1,085,720 records. After exact duplicate rows were removed, 817,006 records remained.

3.4 Train-Test Split

The cleaned working dataset was divided into training and testing data using an 80/20 split. Instead of using a normal random split, GroupShuffleSplit from scikit-learn was used.

The 39 input features were used to group records with identical feature combinations. Records with the same feature values were kept entirely in either the training set or the testing set. This was done to reduce the risk of data leakage.

The final split contained 653,610 training records and 163,396 testing records, with all 34 classes present in both datasets.

3.5 Machine-Learning Models

This study assessed four machine-learning models: LightGBM, Random Forest, Decision Tree, and Logistic Regression.

Logistic Regression was used as a simple baseline model. Decision Tree was included because it is easy to interpret and has relatively low computational complexity. Random Forest was used to examine whether combining multiple decision trees could improve detection performance. LightGBM was included as a gradient-boosting model that is designed to work efficiently with structured data.

3.6 Class-Imbalance Handling

Three main imbalance-handling conditions were tested in this study: no balancing, class weighting and SMOTE. Random undersampling was also tested as an additional experiment with the Decision Tree model.

Class weighting gives more importance to mistakes made on minority classes without changing the number of training records. SMOTE creates synthetic minority-class samples to make the training data more balanced. Random undersampling reduces the number of majority-class samples.

These imbalance-handling methods were applied only to the training data. The testing data was kept unchanged so that the models could be evaluated fairly on the same test set.

3.7 Model Evaluation

Precision, recall, F1-score, and macro F1-score were used to assess the models. Because the macro F1-score assigns equal weight to each class, it is helpful in cases where the dataset is extremely unbalanced.

A separate average F1-score was also calculated for the 10 rare attack classes identified earlier. This was used to focus specifically on how well the models detected rare cyberattacks instead of relying only on overall performance.

3.8 Computational Efficiency

To evaluate whether the models were lightweight, three computational measures were recorded: training time, inference time and saved model size.

Training time measured how long each model took to learn from the training data. Inference time measured how long the trained model took to make predictions on the test dataset. Model size was measured from the saved model file in megabytes.

The trade-off between computational efficiency and rare-attack detection performance was examined by comparing these metrics with the detection outcomes.

3.9 Experimental Setup

The experiments were implemented in Python using pandas, scikit-learn, imbalanced-learn and LightGBM. A fixed random state of 42 was used where supported so that the experiments could be reproduced consistently.

Logistic Regression used StandardScaler before model training. Decision Tree, Random Forest and LightGBM were trained without feature scaling because they are tree-based models. The same training and testing split was used for all models so that the results could be compared fairly.

4. Results

4.1 Overall Model Comparison

The experiments compared Logistic Regression, Decision Tree, Random Forest and LightGBM under different class-imbalance handling methods. The main evaluation measures were macro F1-score, rare-class average F1-score, training time, inference time and model size. The results showed that no single model performed best in every area. Random Forest produced the strongest rare-attack detection results, while Logistic Regression produced the smallest models. Decision Tree provided a middle ground between detection performance and computational cost. LightGBM showed relatively weak performance without imbalance handling but improved substantially when class weighting and SMOTE were applied.

Model	Imbalance Method	Macro F1	Rare-class F1	Training Time (s)	Inference Time (s)	Model Size (MB)
Decision Tree	None	0.5911	0.4072	11.03	0.109	93.22
Decision Tree	Class Weighting	0.5892	0.4059	12.15	0.111	99.38
Decision Tree	Random Undersampling	0.5494	0.3679	0.51	0.076	6.20
Decision Tree	SMOTE	0.5919	0.4170	141.16	0.139	193.72
Logistic Regression	None	0.5365	0.3352	687.69	0.132	0.013
Logistic Regression	Class Weighting	0.5379	0.3452	1018.85	0.197	0.013
Logistic Regression	SMOTE	0.5358	0.3424	840.06	0.166	0.013
Random Forest	None	0.6331	0.4580	29.70	1.320	4536.74
Random Forest	Class Weighting	0.6375	0.4655	25.11	1.258	4130.90
Random Forest	SMOTE	0.6341	0.4663	378.61	25.818	14319.57
LightGBM	None	0.2342	0.0791	42.00	12.443	10.96
LightGBM	Class Weighting	0.4044	0.3445	39.48	11.644	11.25
LightGBM	SMOTE	0.5798	0.4167	218.15	17.784	11.38

Table 1. Overall comparison of model performance and computational efficiency.

4.2 Rare Attack Detection Performance

Random Forest achieved the strongest rare-class detection results. The baseline Random Forest obtained a rare-class average F1-score of 0.4580, while class weighting increased this to 0.4655. SMOTE produced the highest Random Forest rare-class F1-score of 0.4663. However, the improvement from class weighting to SMOTE was very small.

Decision Tree achieved a rare-class F1-score of 0.4072 without balancing. SMOTE improved this to 0.4170, while random undersampling reduced the rare-class F1-score to 0.3679.

Logistic Regression produced lower rare-class performance. Its baseline rare-class F1-score was 0.3352, class weighting improved it to 0.3452, and SMOTE produced 0.3424.

LightGBM showed the largest change after imbalance handling. Its baseline rare-class F1-score was only 0.0791. Class weighting increased this to 0.3445, while SMOTE increased it further to 0.4167.

Table 2. F1-scores for the 10 rare attack classes across selected model configurations.

Rare attack class	Logistic Regression + Class Weighting	Random Forest + Class Weighting	Random Forest + SMOTE	LightGBM + SMOTE
DDOS-HTTP_FLOOD	0.8305	0.9690	0.9667	0.8405
DDOS-SLOWLORIS	0.9158	0.9619	0.9658	0.8504
DICTIONARYBRUTEFORCE	0.4400	0.5874	0.6090	0.5952
BROWSERHIJACKING	0.2916	0.5408	0.5230	0.4920
COMMANDINJECTION	0.2533	0.4070	0.3967	0.3245
SQLINJECTION	0.3156	0.3263	0.3345	0.3231
XSS	0.0685	0.2392	0.2346	0.2413
BACKDOOR_MALWARE	0.0962	0.2413	0.2443	0.2240
RECON-PINGSWEEP	0.1511	0.2840	0.2634	0.1587
UPLOADING_ATTACK	0.0889	0.0981	0.1253	0.1171

Table 2 shows that performance varied considerably between rare attack classes, with DDOS-HTTP_FLOOD and DDOS-SLOWLORIS generally being detected more successfully than very small classes such as UPLOADING_ATTACK.

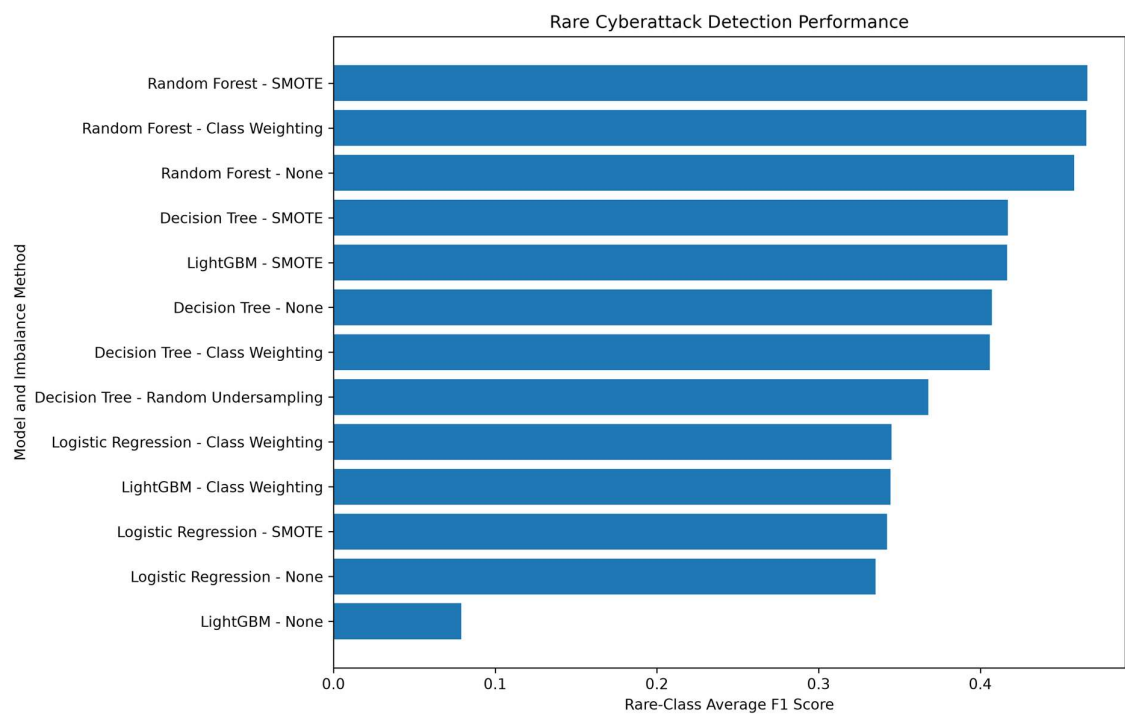


Figure 1. Rare-class average F1-score across model and imbalance-handling configurations.

4.3 Computational Efficiency

The models showed large differences in computational efficiency. Logistic Regression produced extremely small, saved models of approximately 0.013 MB, although its training time was relatively high. Decision Tree models were larger, ranging from approximately 6.2 MB with random undersampling to 193.7 MB with SMOTE.

Random Forest required substantially more storage than the other models. Its baseline model size was approximately 4.54 GB, class weighting produced approximately 4.13 GB, and SMOTE increased the model size to approximately 14.32 GB. The Random Forest with SMOTE also had a much higher inference time of approximately 25.82 seconds.

LightGBM produced models of only around 11 MB in all three configurations. However, its inference time was higher than Decision Tree and Logistic Regression in these experiments.

Table 3. Performance and efficiency of selected configurations from each model family.

Model configuration	Macro F1	Rare-class F1	Training Time (s)	Inference Time (s)	Model Size (MB)
Decision Tree + SMOTE	0.5919	0.4170	141.16	0.139	193.72
Logistic Regression + Class Weighting	0.5379	0.3452	1018.85	0.197	0.013
Random Forest + Class Weighting	0.6375	0.4655	25.11	1.258	4130.90
LightGBM + SMOTE	0.5798	0.4167	218.15	17.784	11.38

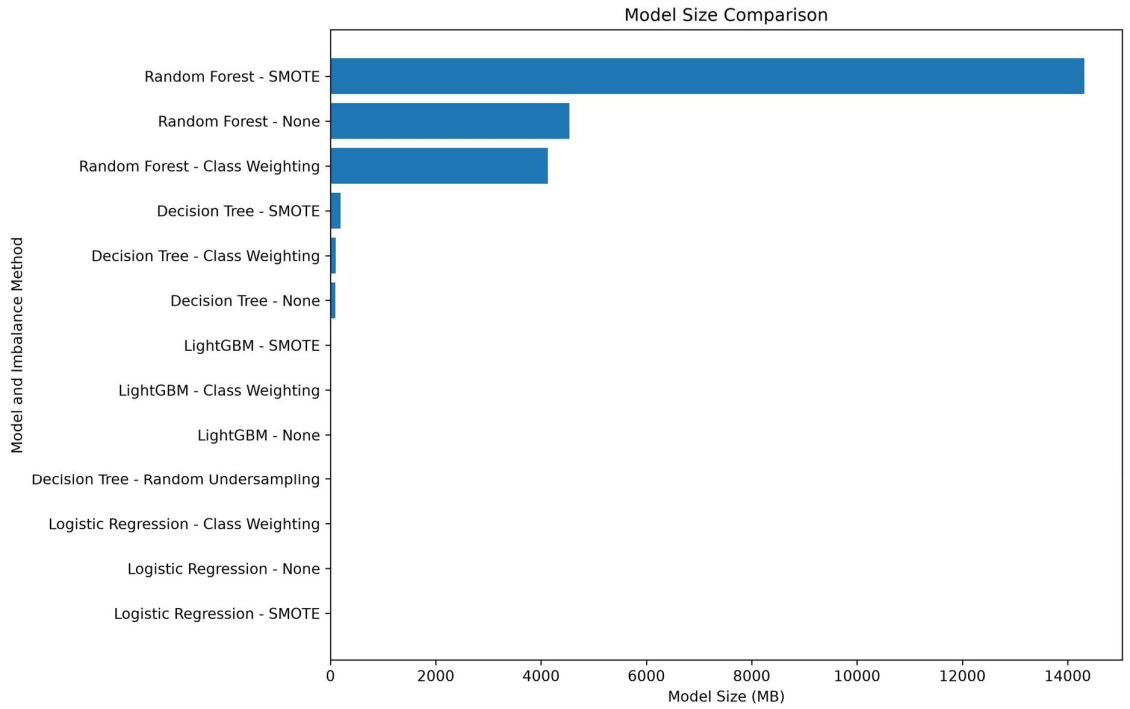


Figure 2. Saved model size across model and imbalance-handling configurations.

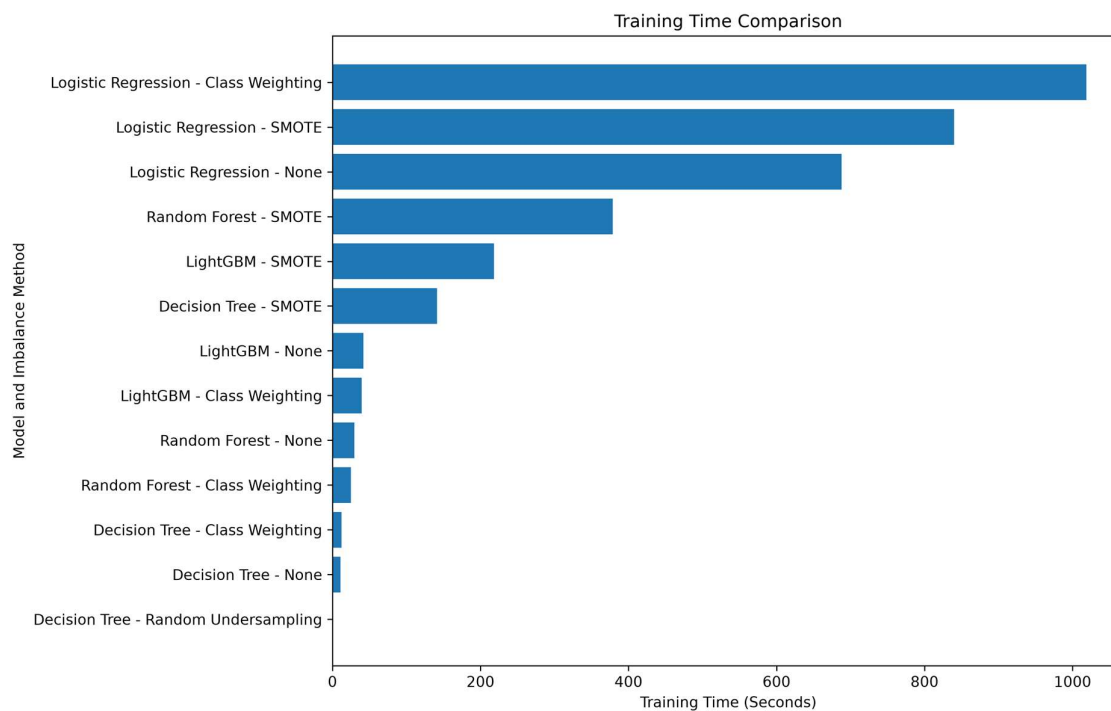


Figure 3. Training time across model and imbalance-handling configurations.

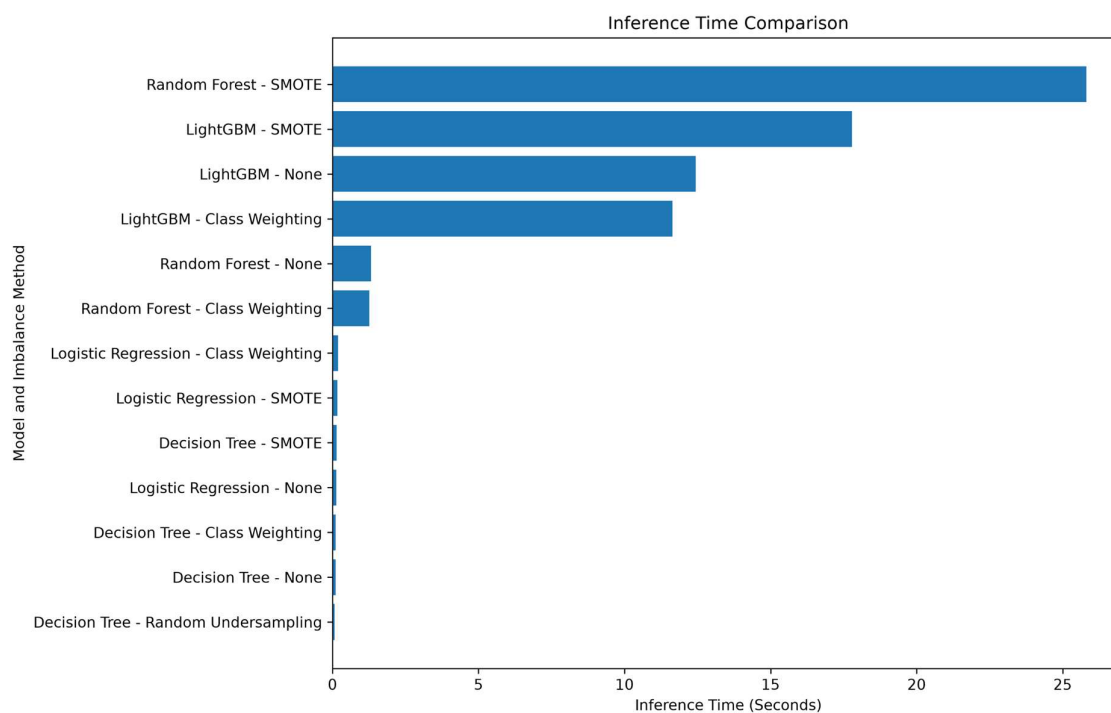


Figure 4. Inference time across model and imbalance-handling configurations.

4.4 Performance-Efficiency Trade-off

The results show a clear trade-off between rare-attack detection performance and computational efficiency. Random Forest achieved the strongest rare-class F1-scores, but required very large model sizes, particularly when combined with SMOTE. LightGBM with SMOTE achieved a rare-class F1-score of 0.4167 while using a model of only about 11.38 MB. Decision Tree with SMOTE achieved a very similar rare-class F1-score of 0.4170, but used a larger model of about 193.72 MB. Logistic Regression was by far the smallest model, but its rare-class detection performance was lower.

5. Discussion

5.1 Effect of Class Imbalance Handling

The experiments showed that imbalance-handling methods affected the models differently. Class weighting produced only small improvements for Logistic Regression and Random Forest, but it greatly improved LightGBM compared with its baseline result. SMOTE also improved rare-class detection for Decision Tree, Random Forest and LightGBM, although the improvement was not always large enough to justify the additional computational cost.

For Random Forest, class weighting and SMOTE produced very similar rare-class F1-scores of 0.4655 and 0.4663 respectively. However, SMOTE increased training time and model size considerably. This suggests that class weighting may be a more computationally practical option for Random Forest in this experiment.

LightGBM responded differently. Its baseline rare-class F1-score was very low, but both class weighting and SMOTE improved performance substantially. SMOTE gave the strongest LightGBM rare-class result, showing that imbalance handling was especially important for this model.

5.2 Rare Attack Detection

The results also showed that the rarest attacks remained difficult to detect. Some classes, such as DDOS-HTTP-FLOOD and DDOS-SLOWLORIS, achieved relatively strong F1-scores across several models. In contrast, very small classes such as UPLOADING_ATTACK, RECON-PINGSWEEP and BACKDOOR_MALWARE generally produced much lower results.

This supports the main motivation of the research: overall or majority-class performance alone is not enough when evaluating intrusion-detection models on highly imbalanced IoT traffic. Looking at rare-class F1-scores separately gave a clearer picture of how well the models handled minority attacks.

5.3 Lightweight Performance Trade-off

The experiments showed that stronger detection performance did not always mean better computational efficiency. Random Forest produced the highest rare-class F1-

scores, but its saved models were several gigabytes in size. The SMOTE version reached approximately 14.32 GB, making it considerably heavier than the other models.

Logistic Regression showed the opposite behaviour. Its saved model was extremely small at approximately 0.013 MB, but its rare-class detection performance was weaker. Decision Tree provided moderate detection performance with lower inference time, while LightGBM maintained a small model size of about 11 MB and improved strongly after imbalance handling.

These results show why multiple efficiency measures are needed when describing a model as lightweight. A model may be small in storage size but slow to train or infer, while another model may detect attacks well but require much more storage.

5.4 Main Findings in Relation to the Research Questions

For RQ1, the results showed that lightweight machine-learning models had different levels of success in detecting rare cyberattacks. Random Forest achieved the strongest rare-class detection, while Logistic Regression produced the smallest models.

For RQ2, imbalance-handling methods generally improved rare-class detection, but their effect depended on the model. Class weighting was particularly useful for Random Forest and LightGBM, while SMOTE produced the strongest rare-class results for Decision Tree and LightGBM.

For RQ3, the experiments showed a clear trade-off between detection performance and computational efficiency. Higher rare-class F1-scores sometimes required larger models or longer training and inference times. Therefore, the most suitable model depends on whether detection performance, storage size, training cost or inference speed is the main priority.

5.5 Limitations

The study has several limitations. First, the full CICIoT2023 dataset contains more than 45 million records, so a smaller working dataset was created for computational feasibility. Rare classes were preserved, meaning the class distribution in the working dataset does not exactly match the original dataset.

Second, the rare-class threshold of below 0.1% was used as an operational definition for this study and may require further validation from the literature or supervisor feedback.

Third, the experiments used a single train-test split and single timing measurements rather than repeated cross-validation. Training and inference times can therefore vary depending on the computer and system load.

Finally, standard SMOTE was applied to the training data. Because SMOTE uses distances between samples, differences in feature scale and the presence of discrete

or binary-like features may affect the synthetic samples that are generated. This should be considered when interpreting the SMOTE results.

6. Conclusion and Future Work

This research evaluated four lightweight machine-learning models for detecting rare cyberattacks in imbalanced IoT network traffic using the CICIoT2023 dataset. Logistic Regression, Decision Tree, Random Forest and LightGBM were compared under different imbalance-handling methods, including no balancing, class weighting and SMOTE. Random undersampling was also tested with Decision Tree.

The results showed that there is no single model that is strongest in every area. Random Forest achieved the highest rare-class detection performance, but its model size was much larger than the other approaches. Logistic Regression produced extremely small models but weaker rare-class performance. Decision Tree provided a moderate balance between performance and efficiency. LightGBM improved substantially when imbalance handling was applied and maintained a relatively small model size.

The experiments also showed that imbalance handling can improve the detection of rare attacks, although the benefit depends on the model. In particular, class weighting gave almost the same rare-class performance as SMOTE for Random Forest while requiring much lower computational cost. SMOTE produced strong improvements for LightGBM but increased training and inference time.

Overall, the findings show that evaluating rare-attack detection alone is not enough when selecting a model for IoT intrusion detection. Computational efficiency also needs to be considered, especially for resource-constrained environments.

Future work could evaluate the models using repeated train-test runs or cross-validation to improve the reliability of the results. Additional imbalance-handling methods could also be investigated, along with feature selection or dimensionality reduction to reduce computational cost. The rare-class threshold could be further validated using previous research, and the models could be tested on another IoT intrusion-detection dataset to examine whether the findings generalise beyond CICIoT2023.

References

- Almahaqeri, S. A., Almourish, M. H., Nasser, A. A., Alghawli, A. S. A., Elsayed, A. A. K., & Alhejoj, A. N. (2026). An optimized gradient boosting framework for IoT intrusion detection: A comprehensive evaluation on the CICIoT2023 dataset. *Scientific Reports*, 16, 16909.
- Anbiaee, Z., Dadkhah, S., & Ghorbani, A. A. (2025). FIGS: A realistic intrusion-detection framework for highly imbalanced IoT environments. *Electronics*, 14(14), 2917.

Ismail, S., Dandan, S., & Qushou, A. (2025). Intrusion detection in IoT and IIoT: Comparing lightweight machine learning techniques using TON_IoT, WUSTL-IIOT-2021, and EdgelloTset datasets. *IEEE Access*, 13, 73468–73485.

Neto, E. C. P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., & Ghorbani, A. A. (2023). CIIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment. *Sensors*, 23(13), 5941.

Yang, Y., Gu, Y., & Yan, Y. (2023). Machine learning-based intrusion detection for rare-class network attacks. *Electronics*, 12(18), 3911.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, 3149–3157.

Chicco, D., & Jurman, G. (2020). The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21, 6.